

AI / ML COMPLETE INTERVIEW REVISION NOTES

[!NOTE]

How to use these notes: This revision sheet is structured specifically for **AI/ML technical interviews**. Every concept is explained from first principles in clear, simple language with intuition, examples, pros/cons, and key interview distinctions.

Terms marked with ★ are important high-yield additions.

1. AI / ML BIG PICTURE

Artificial Intelligence (AI)

- **What it is:** The overarching field of computer science dedicated to building machines and software capable of performing tasks that normally require human intelligence.
- **Examples:** Understanding language, recognizing objects in images, playing strategic games, making diagnoses, generating creative content.
- **Interview Summary:** *"AI is the broadest umbrella discipline concerned with creating intelligent systems that can perceive, reason, learn, and act."*

Machine Learning (ML)

- **What it is:** A specialized subset of AI where systems automatically learn patterns and rules directly from data, rather than having every rule explicitly hardcoded by a software engineer.
- **How it works:** Instead of writing rules manually, you feed the algorithm thousands of historical examples (inputs + outcomes). The algorithm extracts the mathematical relationships and builds a model to predict outcomes on new, unseen data.
- **Example:** Instead of hand-crafting 1,000 keyword rules for an email spam filter ("if contains 'free cash' → spam"), you show the algorithm 50,000 spam and non-spam emails and let it learn the distinguishing patterns on its own.

Deep Learning (DL)

- **What it is:** A specialized subset of Machine Learning based on Artificial Neural Networks with multiple intermediate layers ("deep" architectures).
 - **Why it matters:** Traditional ML requires human engineers to manually craft features (e.g., measuring circle roundness or edge count). Deep learning automatically learns hierarchical feature representations directly from raw, unstructured data (pixels, audio waves, text tokens).
 - **Example:** Facial recognition, autonomous vehicle perception, Large Language Models.
-

Generative AI (GenAI)

- **What it is:** A branch of AI focused on **creating brand-new, original content** (text, images, audio, synthetic data, code) rather than solely analyzing, categorizing, or predicting numbers from existing data.
 - **How it works:** It learns the underlying statistical distribution of massive training datasets and generates novel samples that mirror the style and structure of the training data.
 - **Examples:** ChatGPT (text), Midjourney / DALL-E (images), Suno (music), GitHub Copilot (code).
-

Natural Language Processing (NLP)

- **What it is:** The domain of AI focused on giving computers the ability to read, understand, interpret, translate, and generate human language.
 - **Examples:** Machine translation (Google Translate), sentiment analysis, spam classification, document summarization, voice assistants.
-

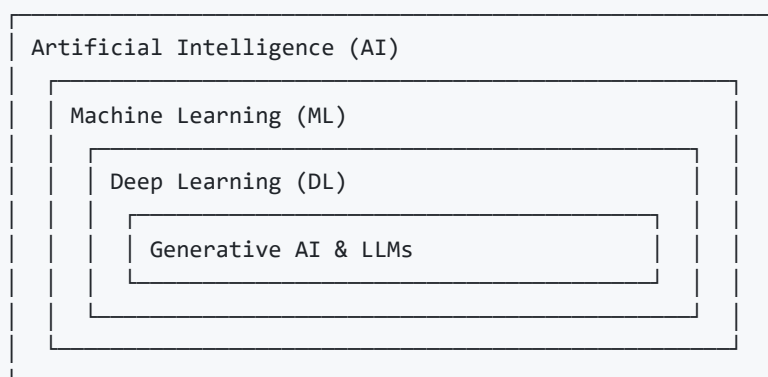
Computer Vision (CV)

- **What it is:** The domain of AI that enables computers to extract meaningful information, context, and understanding from digital images and videos.
 - **Examples:** Autonomous driving (detecting pedestrians, lane lines), medical imaging (detecting tumors on CT scans), facial unlock.
-

Large Language Model (LLM)

- **What it is:** A deep-learning model containing billions of parameters (usually built on the Transformer architecture) trained on massive web-scale text corpora to understand and generate human-like language.
 - **Examples:** GPT-4, Claude, Gemini, LLaMA.
-

AI vs ML vs DL Hierarchy



Key Interview Distinction:

"All Deep Learning is Machine Learning, and all Machine Learning is AI."*

"Not all AI is ML (e.g., traditional rule-based expert systems), and not all ML is DL (e.g., Linear Regression, Decision Trees)."

Traditional Programming vs Machine Learning

Dimension	Traditional Programming	Machine Learning
Inputs	Data + Rules (Code written by human)	Data + Expected Outputs (Labels)
Output	Answers / Results	Model (Learned Rules & Parameters)
At Inference	Execute fixed code on new input	Model + New Data $\rightarrow$ Prediction
Best suited for	Clear, deterministic business logic	Complex patterns too intricate to write by hand

Algorithm

- **What it is:** A step-by-step mathematical procedure or set of instructions used to solve a problem or learn patterns from data.
- **Normal algorithm:** A programmer defines every exact logic step (e.g., QuickSort, Binary Search).
- **ML algorithm:** A mathematical optimization procedure (e.g., Gradient Descent) that automatically adjusts internal weights to fit data.

2. BASIC DATA TERMINOLOGY

Core Data Concepts

- **Data:** Raw facts, figures, text, audio, or images fed into an AI system.
- **Dataset:** A structured collection of data organized for training, validating, or testing an ML model.
- **Sample / Instance / Row / Record:** A single individual observation in a dataset (e.g., one patient's record, one house's specs).
- **Feature / Input Variable (\$X\$):** An individual measurable property or characteristic used by the model to make predictions (e.g., square footage, number of bedrooms).
- **Label / Target / Ground Truth (\$y\$):** The actual correct answer or outcome we want the model to learn to predict (e.g., actual selling price of the house).

Feature vs Label:

* **Feature (\$X\$)** = The input / question / evidence.

* **Label (\$y\$)** = The true answer / output.

Data Types & Structures

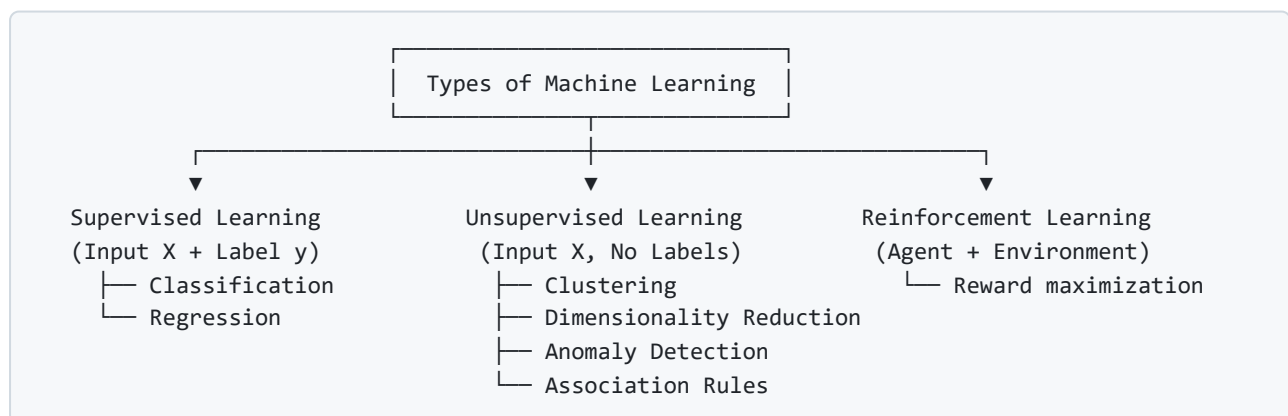
- **Structured Data:** Highly organized data formatted cleanly into rows and columns (e.g., SQL tables, Excel spreadsheets).
- **Unstructured Data:** Data with no predefined tabular format (e.g., raw text documents, audio clips, video, images). Deep Learning is specifically designed to excel here.
- **★ Semi-Structured Data:** Data containing internal markers, tags, or hierarchies without a rigid tabular schema (e.g., JSON, XML, HTML files).
- **★ Tabular Data:** A 2D matrix of rows (samples) and columns (features). The bread and butter of classical ML (Random Forest, XGBoost).

- ★ **Corpus:** A large, curated collection of textual documents used for NLP and LLM pretraining (e.g., all of Wikipedia + Common Crawl).
- ★ **Ground Truth:** The verified real-world fact or true label against which model predictions are scored.
- ★ **Noise:** Random errors, corruption, or irrelevant variations in data that obscure the true underlying signal.

Critical Data Challenges

- ★ **Class Imbalance:**
 - **What it is:** When one category in the target variable heavily outnumbers another (e.g., 99.9% legitimate credit card transactions, 0.1% fraud).
 - **Interview Danger: Accuracy is completely misleading on imbalanced datasets.** A naive model that blindly predicts "Not Fraud" 100% of the time achieves 99.9% accuracy while catching zero fraudulent transactions. Use **Precision, Recall, F1-Score, or PR-AUC** instead.
- ★ **Curse of Dimensionality:**
 - **What it is:** As the number of features (dimensions) grows, the volume of the feature space increases exponentially, causing data points to become extremely sparse.
 - **Why it matters:** Distance metrics (like Euclidean distance in KNN) become meaningless because all points become roughly equidistant. Solved via dimensionality reduction (PCA) or feature selection.
- ★ **Correlation vs Causation:**
 - **Correlation:** A statistical relationship where two variables change together (when X increases, Y tends to increase).
 - **Causation:** A true cause-and-effect relationship where changing X directly causes Y to change.
 - **Core Rule: Correlation does NOT imply causation.**
 - **Example:** Ice cream sales and drowning rates both increase in summer (strong positive correlation). Eating ice cream does *not* cause drowning; the hidden confounding factor is warm weather.

3. TYPES OF MACHINE LEARNING



1. Supervised Learning

- **What it is:** The model learns a predictive mapping from inputs to outputs using **labeled training data** (Input X + Correct Label y).
- **How it works:** The model predicts an outcome, measures its error against the true label, and adjusts its internal parameters to minimize future errors.
- **Main Tasks:**

- **Classification:** Predicting discrete categories (e.g., "Spam" vs "Not Spam").
 - *Binary Classification:* Exactly 2 classes.
 - *Multi-class Classification:* 3+ mutually exclusive classes (sample belongs to exactly 1).
 - ★ *Multi-Label Classification:* A sample can simultaneously belong to multiple classes (e.g., a movie tagged as both "Action" AND "Sci-Fi").
 - **Regression:** Predicting continuous numerical values (e.g., House Price = ₹85.5 Lakh, Temperature = 31.4°C).
-

2. Unsupervised Learning

- **What it is:** Finding hidden patterns, natural groupings, or internal structures in data **without any human-provided labels**.
 - **Main Tasks:**
 - **Clustering:** Grouping similar data points together without pre-existing labels.
 - *K-Means:* Partitions data into K non-overlapping clusters around centroids.
 - ★ *Elbow Method:* A technique to find the optimal K by plotting inertia (distortion) against K and looking for the "elbow bend."
 - ★ *Silhouette Score:* Measures how well-separated clusters are (range: -1 to +1; higher is better).
 - *Hierarchical Clustering:* Builds a tree-like hierarchy of clusters (visualized as a **Dendrogram**).
 - **Dimensionality Reduction:** Compressing high-dimensional feature spaces while retaining maximum variance (e.g., **PCA**, **t-SNE**).
 - **Anomaly / Outlier Detection:** Identifying rare observations that deviate substantially from normal behavior (e.g., credit card fraud detection).
 - **Association Rule Learning:** Uncovering rules that describe large portions of data (e.g., Market Basket Analysis: "Customers who buy bread also buy butter").
-

3. ★ Semi-Supervised Learning

- **What it is:** Training an algorithm on a **small amount of labeled data** combined with a **large volume of unlabeled data**.
 - **Why it matters:** Manual data labeling is expensive and time-consuming. Semi-supervised learning uses the small labeled set to establish initial decision boundaries, then uses the unlabeled data to refine them.
-

4. Reinforcement Learning (RL)

- **What it is:** An autonomous **Agent** learns to make a sequence of decisions by interacting with an **Environment** to maximize cumulative **Rewards**.
-

4. REINFORCEMENT LEARNING TERMINOLOGY

Term	Simple Definition	Example (Chess AI)
Agent	The decision-maker or AI learner	The chess engine
Environment	The world/system the agent interacts with	The chessboard, pieces, and game rules
State (\$s\$)	The current snapshot or situation of the environment	Current positions of all pieces on the board
Action (\$a\$)	A move or decision available to the agent	Moving Queen to E4
Reward (\$r\$)	Immediate numerical feedback for an action	+1 for winning, -1 for losing, 0 for neutral
Policy (\$\pi\$)	The strategy/rulebook mapping states to actions	Strategy: "If king is threatened, protect it"

- ★ **Exploration vs Exploitation:**
 - **Exploration:** Trying new, untested actions to discover better long-term strategies.
 - **Exploitation:** Choosing the best-known action based on current knowledge to secure guaranteed rewards.
 - **The Trade-off:** Pure exploitation leads to getting stuck in suboptimal routines; pure exploration wastes resources on bad moves.

5. DATA PREPROCESSING

Data Cleaning & Transformation

- **Data Preprocessing:** Cleaning and formatting raw data into a reliable numerical representation for ML algorithms.
- **Missing Value Handling:**
 - *Drop rows/columns:* Use only when missing data is negligible (<2%).
 - *Mean/Median Imputation:* Fill numerical gaps (use median when outliers exist).
 - *Mode Imputation:* Fill categorical gaps with the most frequent value.
 - *Model-based Imputation:* Predict missing values using KNN or regression.
- ★ **Outlier Handling:** Identifying extreme, distorting values using Z-scores or IQR, then trimming or capping (winsorization) them.

Categorical Encoding

Nominal (No Order: Colors)	→ One-Hot Encoding	(Red=[1,0,0], Blue=[0,1,0])
Ordinal (Natural Order: Sizes)	→ Ordinal Encoding	(Small=0, Med=1, Large=2)
Target Classes (Labels only)	→ Label Encoding	(Cat=0, Dog=1)

- ★ **Label Encoding:** Assigns each category a sequential integer (0, 1, 2). *Warning:* Algorithms may assume artificial mathematical ordering (2 > 0).
- ★ **One-Hot Encoding:** Creates a separate binary column (0 or 1) for each unique category. *Best for unordered categories.*

- ★ **Ordinal Encoding:** Assigns integers specifically respecting a true natural hierarchy (e.g., Low=0, Medium=1, High=2).

Feature Scaling

- **Why it matters:** Distance-based algorithms (KNN, SVM, K-Means) and Gradient Descent compute distances between features. If Salary is ₹1,00,000 and Age is 25, Salary will dominate calculations purely due to magnitude.
- **Normalization (Min-Max Scaling):** Scales values into a strict range $[0, 1]$. Sensitive to extreme outliers.

$$\text{Normalized } x = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

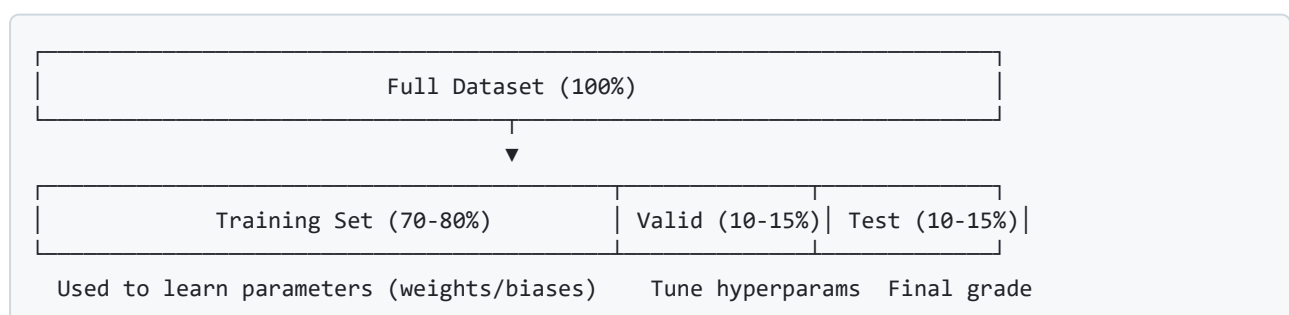
- **Standardization (Z-score Scaling):** Rescales data to have $\text{Mean } \mu = 0$ and $\text{Standard Deviation } \sigma = 1$. Much more robust to outliers.

$$z = \frac{x - \mu}{\sigma}$$

Advanced Preprocessing

- **Feature Engineering:** Using domain expertise to create informative new features from raw inputs (e.g., calculating Age from Date_of_Birth).
- ★ **Feature Selection:** Selecting only the most relevant, non-redundant features to reduce model complexity and prevent overfitting.
- ★ **Data Augmentation:** Artificially increasing dataset diversity by applying transformations (e.g., image flipping, rotation, text synonym replacement).
- ★ **SMOTE (Synthetic Minority Over-sampling Technique):** Synthesizing realistic minority class samples via feature interpolation to resolve severe class imbalances.
- **Data Leakage (Critical Concept):**
 - **What it is:** When information from outside the training dataset (such as the validation/test set or future information) accidentally enters the training pipeline.
 - **Consequence:** Superhuman training performance that catastrophically collapses in production.
 - **Golden Rule:** Always split into Train/Test BEFORE performing any preprocessing (scaling, imputation, encoding).

6. TRAIN / VALIDATION / TEST SPLIT



- **Training Data:** Data the model directly learns parameters (weights) from.
- **Validation Data:** Holdout data used during development to tune hyperparameters, select model architectures, and detect overfitting.
- **Test Data:** Completely untouched holdout data used **only once** at the very end to evaluate true real-world generalization.

- ★ **K-Fold Cross-Validation:** The dataset is split into K equal subsets. The model trains on $K-1$ subsets and validates on the remaining 1 subset, repeated K times so every fold acts as the validation set once. Scores are averaged.
- ★ **Stratified K-Fold:** A version of K-Fold that guarantees each fold contains the exact same class percentage ratio as the original dataset (vital for imbalanced data).
- **Generalization:** The ultimate goal of ML — the model's ability to make accurate predictions on completely new, unseen data.

7. MODEL / PARAMETERS / HYPERPARAMETERS

Parameter vs Hyperparameter

Dimension	Parameter	Hyperparameter
Where it comes from	Learned automatically from training data	Set manually by the human engineer
When it is set	Updated during model training	Configured <i>before</i> training begins
Examples	Neural network weights & biases; regression slope & intercept	Learning rate, batch size, epochs, tree depth, K in KNN
How to optimize	Solved via Gradient Descent / Loss minimization	Found via Grid Search or Random Search

- ★ **Hyperparameter Tuning:** Systematic search for the best hyperparameter configuration.
 - *Grid Search:* Exhaustively tries every single specified combination (thorough, but slow).
 - *Random Search:* Randomly samples configurations from a distribution (faster, often finds equally good results).
- ★ **Baseline Model:** A simple heuristic or naive model used as a benchmark (e.g., always predicting the majority class or the average price). If a complex deep learning model cannot beat the baseline, it is useless.

8. LOSS / OPTIMIZATION

Loss vs Metric

- **Loss Function:** A mathematical equation measuring error for a **single sample**. Must be smooth and differentiable so optimization algorithms can compute gradients.
 - **Cost Function:** The average loss across the **entire dataset**.
 - **Objective Function:** The general function the optimization algorithm seeks to minimize (loss + regularization) or maximize (reward).
 - **Metric:** A human-interpretable performance indicator (e.g., Accuracy %, F1-Score). Does *not* need to be differentiable.
-

Common Loss Functions

Task Type	Loss Function	Mathematical Concept		
Regression	MSE (Mean Squared Error / L2 Loss)	Averages squared errors $\frac{1}{n} \sum (y - \hat{y})^2$; heavily penalizes outliers.		
Regression	MAE (Mean Absolute Error / L1 Loss)	Averages absolute errors $\frac{1}{n} \sum y - \hat{y} $	$y - \hat{y}$	\$; robust to outliers.
Binary Classification	Binary Cross-Entropy (Log Loss)	Penalizes confident incorrect probabilistic predictions.		
Multi-class Classification	Categorical Cross-Entropy	Generalized cross-entropy across C mutually exclusive classes.		

Gradient Descent Intuition

- **Gradient:** A vector of partial derivatives pointing in the direction of steepest increase of the loss function.
- **Intuition:** If you are blindfolded on a foggy mountain, the gradient tells you which way is straight uphill. To reach the bottom valley (minimum loss), you take steps in the **exact opposite direction (negative gradient)**.
- **Update Formula:**

$$\text{New Weight} = \text{Old Weight} - (\text{Learning Rate}) \times \text{Gradient}$$

- **Learning Rate (α):** The step size taken down the loss landscape.
 - *Too high:* Overshoots the minimum and diverges.
 - *Too low:* Takes forever to converge; gets trapped in local minima.
- **★ Gradient Descent Variants:**
 - *Batch GD:* Computes gradient on the *entire dataset* before taking 1 step (stable, but slow).
 - *Stochastic GD (SGD):* Updates weights after *every single sample* (fast, but erratic).
 - *Mini-Batch GD:* Updates weights on small batches (e.g., 32, 64 samples). Standard industry practice.
- **★ Optimizers:**
 - *SGD + Momentum:* Adds momentum from past steps to power through flat regions.
 - *RMSProp:* Adapts individual learning rates per parameter based on gradient magnitudes.
 - *Adam (Adaptive Moment Estimation):* Combines Momentum + RMSProp. The universal default for deep learning.
- **Convergence:** When parameter updates become negligible and the loss function flattens out at its minimum.

9. EPOCH / BATCH / ITERATION

- **Epoch:** Exactly **one complete pass** of the entire training dataset through the model.
- **Batch Size:** The number of training samples processed together before updating weights (e.g., 32, 64).
- **Iteration (Step):** Exactly **one parameter update** step (processing one mini-batch = 1 iteration).

$$\text{Iterations per Epoch} = \frac{\text{Total Training Samples}}{\text{Batch Size}}$$

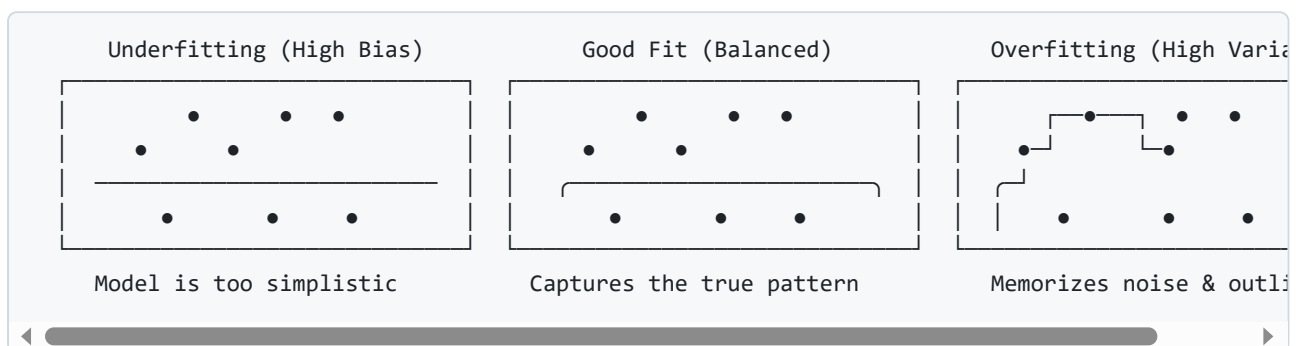
Example:

* 10,000 training images, Batch Size = 100

* 1 Epoch = 100 Iterations (weight updates)

* 25 Epochs = 2,500 total iterations

10. OVERFITTING / UNDERFITTING



- ★ **Model Complexity:** The expressive capacity of a model. Simple straight line = low complexity; deep tree with 1,000 branches = high complexity.
- **Underfitting (High Bias):** Model is too simple to capture patterns. Fails on both Train and Test sets. *Fix:* Increase model complexity, add features, reduce regularization.
- **Overfitting (High Variance):** Model memorizes the training data including noise. Exceptional on Train set, collapses on Test set. *Fix:* Get more data, apply regularization, use dropout, prune trees, use early stopping.
- **Bias-Variance Tradeoff:**
 - As complexity increases, Bias decreases but Variance increases.
 - Total Error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$. The sweet spot balances both.

Regularization Techniques

- **L1 Regularization (Lasso):** Adds penalty $\lambda \sum |w|$. Drives irrelevant feature weights to **exactly zero** (acts as automatic feature selection).
- **L2 Regularization (Ridge):** Adds penalty $\lambda \sum w^2$. Shrinks weights close to zero without making them zero (prevents any single feature from dominating).
- ★ **Elastic Net:** Combines L1 and L2 penalties linearly.
- ★ **Dropout:** Randomly deactivates a percentage of neurons (e.g., 20-50%) during each training pass in neural networks, preventing co-adaptation.
- ★ **Early Stopping:** Automatically halts training when validation loss stops improving and starts rising.
- ★ **Batch Normalization:** Normalizes layer inputs across each mini-batch, stabilizing and accelerating deep network training.

11. ML EVALUATION METRICS

Confusion Matrix

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP) ✓	False Negative (FN) ✗ (Type II Error / Miss)
Actually Negative	False Positive (FP) ✗ (Type I Error / False Alarm)	True Negative (TN) ✓

Classification Formulas & When to Use

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Percentage of all predictions that were correct. Misleading on imbalanced datasets.

- **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Out of everything predicted Positive, what fraction was truly Positive?

- **When to prioritize:** When False Positives are expensive (e.g., Spam filter — don't send important emails to spam).

- **Recall (Sensitivity / True Positive Rate - TPR):**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Out of all actual Positives, what fraction did the model catch?

- **When to prioritize:** When False Negatives are dangerous (e.g., Cancer detection, fraud detection — you cannot miss a true positive).

- **F1-Score:**

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Harmonic mean balancing Precision and Recall. Best single metric for imbalanced data.

- **Specificity (True Negative Rate - TNR):** $\frac{TN}{TN + FP}$ (Fraction of true negatives correctly identified).
- ★ **False Positive Rate (FPR):** $\frac{FP}{FP + TN} = 1 - \text{Specificity}$ (Fraction of true negatives falsely flagged).

★ Classification Threshold & Precision-Recall Trade-off

- Classification models output probabilities (e.g., 0.78). A **Threshold** (default 0.50) turns that probability into a class label.
- **Lowering the Threshold (e.g., 0.20):** Model flags positives aggressively. **Recall increases** (catches more cases), but **Precision decreases** (more false alarms).
- **Raising the Threshold (e.g., 0.80):** Model flags positives conservatively. **Precision increases** (fewer false alarms), but **Recall decreases** (misses cases).

ROC Curve & AUC

- **ROC Curve:** Plots **True Positive Rate (Recall)** on the Y-axis against **False Positive Rate (FPR)** on the X-axis across all possible thresholds.

- **AUC (Area Under Curve):** A single number from 0.0 to 1.0 measuring class separation ability (\$0.5 = \text{random guessing}\$, \$1.0 = \text{perfect}\$).

★ Regression Metrics Summary

Metric	Formula	Interview Explanation	
MAE	$\frac{1}{n} \sum y - \hat{y} $	Mean Absolute Error. Direct error in original units; robust to outliers.	\$
MSE	$\frac{1}{n} \sum (y - \hat{y})^2$	Mean Squared Error. Heavily penalizes large errors.	
RMSE	$\sqrt{\text{MSE}}$	Root Mean Squared Error. In original units; sensitive to large errors.	
R^2 Score	$1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$	Proportion of variance in target explained by the model (\$1.0 = \text{perfect}\$, \$0.0 = \text{predicts mean}\$).	

12. MAJOR ML ALGORITHMS

1. Linear Regression

- **What it is:** Fits a straight linear equation $y = w_1x_1 + w_2x_2 + \dots + b$ to predict a continuous target.
- **Pros:** Simple, highly interpretable, extremely fast.
- **Cons:** Assumes strict linear relationship; sensitive to outliers.
- **When to use:** Continuous numerical prediction with linear patterns.

2. Logistic Regression

- **What it is:** A classification algorithm that passes a linear combination of features through the **Sigmoid function** to output probabilities between 0 and 1.
- **Pros:** Fast, interpretable (coefficients represent odds ratios), outputs calibrated probabilities.
- **Cons:** Assumes linear decision boundaries.
- **When to use:** Baseline binary or multi-class classification.

3. K-Nearest Neighbors (KNN)

- **What it is:** A non-parametric, lazy algorithm that classifies new points based on the majority vote of the K closest training samples.
- **Pros:** Simple, zero training phase, adapts to multi-class problems.
- **Cons:** Very slow inference on large datasets; suffers from the curse of dimensionality; requires feature scaling.
- **When to use:** Small datasets with low feature dimensions.

4. Support Vector Machine (SVM)

- **What it is:** Finds the optimal hyperplane that separates classes with the maximum geometric margin.
 - ★ **Kernel Trick:** Projects non-linear data into higher-dimensional space where it becomes linearly separable, without explicitly computing higher-dimensional coordinates.
 - **Pros:** Effective in high-dimensional spaces; memory-efficient.
 - **Cons:** Slow to train on large datasets; sensitive to noise and hyperparameter tuning.
 - **When to use:** Medium-sized datasets with clear margins of separation, text classification.
-

5. Naive Bayes

- **What it is:** A probabilistic classifier based on Bayes' Theorem that assumes all features are conditionally independent given the class label.
 - **Pros:** Extremely fast, requires minimal training data, handles high dimensions easily.
 - **Cons:** The feature independence assumption is almost never true in real data.
 - **When to use:** Text classification, spam filtering, sentiment analysis baselines.
-

6. Decision Tree

- **What it is:** A tree-structured flowchart model that recursively splits data using if-then rules based on purity criteria.
 - ★ **Splitting Criteria:**
 - *Entropy:* Measures impurity in a node. Entropy = 0 for a pure node. It is highest when classes are evenly distributed. **Maximum value depends on the number of classes ($\log_2(C)$).**
 - *Information Gain:* The drop in entropy achieved by splitting on a feature.
 - *Gini Impurity:* Probability of misclassifying a random sample ($0.0 = \text{pure}$). Faster to compute than entropy.
 - **Pros:** Highly interpretable, visualizable, handles non-linear data, no feature scaling required.
 - **Cons:** Prone to severe overfitting; unstable (small data changes create totally different trees).
 - **When to use:** When human interpretability and rule-based explanations are strictly required.
-

7. Ensemble Learning: Bagging vs Boosting

Dimension	Bagging (e.g., Random Forest)	Boosting (e.g., XGBoost, Gradient Boosting)
How trees are built	In parallel (independently)	Sequentially (one after another)
Training data	Random bootstrap samples of data	Reweighted data focusing on prior errors
Primary goal	Reduces Variance (prevents overfitting)	Reduces Bias (converts weak learners to strong)
Interpretability	Moderate / Feature Importances	Moderate / Feature Importances
Overfitting risk	Very low (hard to overfit)	Can overfit if learning rate is too high

- **Random Forest:** Ensemble of decision trees using Bagging + Feature Randomness. Strong general-purpose tabular baseline.
 - **Gradient Boosting & XGBoost:** Sequentially trains trees to fit the residual errors (loss gradients) of prior trees. SOTA accuracy on tabular data.
-

8. K-Means Clustering & PCA

- **K-Means:** Centroid-based clustering. Fast and scalable, but must specify K manually and assumes spherical clusters.
- **PCA:** Unsupervised linear dimensionality reduction. Maximizes variance along orthogonal components. Great for compressing features, but components lack human interpretability.

13. NEURAL NETWORK FUNDAMENTALS

Inputs (x) $\longrightarrow$ [Multiply by Weights (w)] $\longrightarrow$ [Add Bias (b)] $\longrightarrow$ [Activation Function]

- **Artificial Neuron:** Computes $\text{Output} = \text{Activation}(\sum w_i x_i + b)$.
- **Weights (w):** Learnable parameters determining connection strength and importance.
- **Bias (b):** Learnable parameter shifting the activation threshold independently of inputs.
- **Activation Function:** Introduces **non-linearity** into the network. Without non-linear activations, stacking 100 layers collapses mathematically into a single simple linear regression ($W_2(W_1x) = W_{\text{combined}}x$).
 - **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Default for hidden layers; avoids vanishing gradients for positive inputs.
 - **Leaky ReLU:** $f(x) = \max(\alpha x, x)$. Prevents "Dying ReLU" by allowing a small gradient for negative inputs.
 - **Sigmoid:** Outputs $(0, 1)$. Used for binary classification output layer.
 - **Softmax:** Converts raw logits into a normalized probability distribution across C classes (sums to 1.0). Used in multi-class output layers.

Training Mechanics

- **Forward Propagation:** Data flows forward (Input $\rightarrow$ Hidden $\rightarrow$ Output), applying weights, biases, and activations to produce a prediction.
- **Backpropagation (Key Concept):**
 1. Computes the output loss compared to ground truth.
 2. Applies the **Calculus Chain Rule in reverse** (from output back to input) to calculate the partial derivative (gradient) of the loss with respect to every single weight and bias.
 3. The optimizer updates all parameters in the opposite direction of the gradient to reduce error.

14. TYPES OF NEURAL NETWORKS

- **Feedforward Neural Network (FNN / MLP):** Simple architecture where signals flow strictly forward in one direction with no cycles.
- **Convolutional Neural Network (CNN):** Specialized for spatial grid data (images). Uses sliding convolutional filters to extract feature maps.
- **Recurrent Neural Network (RNN):** Specialized for sequential data (text, time series). Contains internal loops passing a hidden state (memory) over time.

- **LSTM (Long Short-Term Memory):** Solves the vanishing gradient problem of standard RNNs using a dedicated Cell State and 3 gates (Forget, Input, Output).
 - ★ **GRU (Gated Recurrent Unit):** Streamlined LSTM variant with 2 gates (Reset, Update). Trains faster with comparable performance.
 - **Transformer:** Modern architecture based entirely on Self-Attention, processing sequences in parallel without recurrence.
-

15. CNN TERMINOLOGY

- **Convolution:** Sliding a small learnable filter matrix over an image to calculate dot products and extract local feature maps.
 - **Kernel / Filter:** A small matrix (e.g., 3×3) trained to detect visual patterns (edges, textures, shapes).
 - **Feature Map:** The output matrix showing where specific patterns were detected in the image.
 - **Stride:** The step size (number of pixels) the filter moves at each step.
 - **Padding:** Adding border pixels (typically zeros) to preserve spatial dimensions at the edges.
 - **Pooling:** Downsampling feature maps to reduce spatial dimensions, lower compute load, and provide translation invariance (e.g., **Max Pooling** keeps the maximum value in each window).
 - **Channel:** The depth dimension of an image tensor (e.g., standard RGB image has 3 color channels).
-

16. RNN / LSTM TERMINOLOGY

- **Sequential Data:** Ordered data where sequence position carries vital context (sentences, time-series, audio).
 - **Hidden State (h_t):** The memory vector carried forward across time steps in an RNN.
 - **Vanishing Gradient Problem:** In long sequences, multiplying gradients repeatedly across time steps causes them to shrink exponentially toward zero, making standard RNNs forget long-range context.
 - **LSTM Gates:**
 - *Forget Gate:* Decides what useless information to discard from the cell state.
 - *Input Gate:* Decides what new information to store in the cell state.
 - *Output Gate:* Decides what information to emit as the hidden state for the current step.
 - **Long-Term Dependency:** Relationships between tokens separated by large sequence distances (e.g., "*The keys that I left on the kitchen table **were** lost*").
-

17. TRANSFORMER FUNDAMENTALS

★ Why Transformers Replaced RNNs

Dimension	Recurrent Neural Networks (RNN/LSTM)	Transformers
Processing Style	Sequential (word-by-word)	Parallel (all tokens simultaneously)
GPU Utilization	Poor (cannot parallelize sequential steps)	Exceptional (massive GPU cluster scaling)
Long-Range Context	Information decays across time steps ($O(N)$ path)	Direct attention connection between all tokens ($O(1)$ path)
Training Speed	Slow on large text	Orders of magnitude faster

Core Transformer Mechanics

- **Token & Tokenization:** Breaking raw text into atomic sub-words or characters (e.g., "unbreakable" $\rightarrow [\text{"un"}, \text{"break"}, \text{"able"}]$) using tokenizers like BPE or WordPiece.
- **Embeddings:** Dense high-dimensional vectors where semantically and contextually related items share mathematical proximity and similar vector directions.
- **Self-Attention:** An operation where each token in a sequence attends to all other tokens in the same sequence to compute contextual representations.
- **Query, Key, Value (Q, K, V):**
 - *Query (Q):* What the current token is searching for.
 - *Key (K):* What context each token offers to match against.
 - *Value (V):* The actual information content passed forward.

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Multi-Head Attention:** Running multiple self-attention calculations in parallel to capture diverse linguistic relationships (syntax, coreference, semantic meaning).
- **Positional Encoding:** Injected vectors providing token order information, since attention processes all tokens in parallel without inherent sequential awareness.
- **★ Causal Masking:** A triangular mask in decoder models preventing tokens from attending to future tokens during training (when predicting token 3, the model can only see tokens 1 and 2).
- **Residual / Skip Connections:** Adding layer input directly to layer output ($x + \text{SubLayer}(x)$) to ensure unimpeded gradient flow through deep networks.
- **Architectures:**
 - *Encoder-Only (BERT):* Bidirectional understanding; great for classification and Named Entity Recognition.
 - *Decoder-Only (GPT):* Autoregressive generation using causal masking; great for text generation and chat.
 - *Encoder-Decoder (T5, BART):* Sequence-to-sequence transformation; great for translation and summarization.

18. SELF-SUPERVISED LEARNING

- **What it is:** A training paradigm where the model creates its own learning signal from raw, unlabelled data without human annotation.

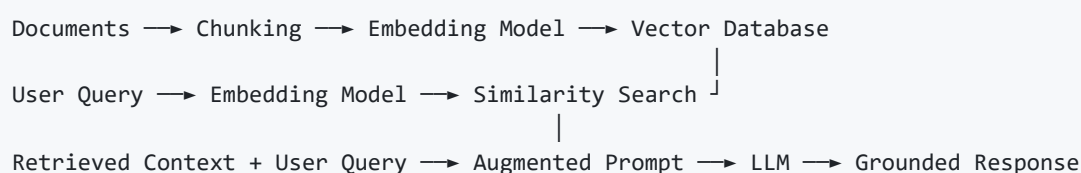
- **How LLM Pretraining Uses It:** The model is given trillions of words from the web and trained to predict missing words (**Masked LM**) or the next token (**Causal LM**).

19. LLM / GENERATIVE AI

- **Large Language Model (LLM):** A deep Transformer trained on massive text data to understand, reason, and generate language.
- ★ **Next-Token Prediction:** The core pretraining task of autoregressive LLMs. Given preceding tokens $[t_1, t_2, \dots, t_n]$, the model calculates a probability distribution to pick token t_{n+1} .
- ★ **Parameter Count:** The total number of trainable weights and biases in the model (e.g., a 7B model has ~7 billion parameters). More parameters provide greater capacity, but data quality and architecture are equally vital.
- **Pretraining:** Training a base model on raw internet-scale data to learn general language and world knowledge.
- **Fine-Tuning:** Further training a pretrained model on a curated task-specific dataset to adapt its behavior and style.
- ★ **PEFT & LoRA (Low-Rank Adaptation):** Fine-tuning only a tiny fraction of parameters ($<1\%$) by injecting small trainable low-rank matrices, drastically cutting GPU memory requirements.
- ★ **RLHF (Reinforcement Learning from Human Feedback):** A post-training alignment method where human preference ratings train a reward model, which then optimizes the LLM via RL (e.g., PPO) to produce helpful, harmless outputs.
- **Prompt Engineering:** Structuring instructions and context to guide LLM outputs effectively.
- **Zero-Shot vs Few-Shot:** Zero-Shot = task with zero examples; Few-Shot = providing 2-5 demonstration examples in the prompt.
- ★ **Chain-of-Thought (CoT):** Asking the model to "think step by step" to generate intermediate reasoning chains, improving math and logic accuracy.
- **Context Window:** The maximum number of tokens a model can process in a single interaction.
- **Hallucination:** When an LLM produces factually false or nonsensical statements while sounding completely confident and plausible.
- ★ **Sampling Parameters:**
 - *Temperature:* Controls randomness ($0.0 = \text{deterministic}$, $1.0 = \text{creative}$).
 - *Top-k:* Limits choices to top k most probable tokens.
 - *Top-p (Nucleus):* Limits choices to the smallest set of tokens reaching cumulative probability p .

20. RETRIEVAL-AUGMENTED GENERATION (RAG)

The RAG Architecture



★ RAG vs Fine-Tuning

Dimension	Retrieval-Augmented Generation (RAG)	Fine-Tuning
What it modifies	Supplies external Information in the prompt context	Updates internal Parameters (weights) of the model
Analogy	Giving the student an open-book exam	Teaching the student a new subject / skill
Updating data	Instant (just add/edit docs in database)	Requires retraining runs
Hallucination	Low (grounded in retrieved citations)	Moderate to High (can still hallucinate)
Best suited for	Dynamic internal knowledge bases, private docs	Specific tone/style, domain jargon, rigid output formats

★ RAG Advantages & Drawbacks

• Advantages:

- Access to real-time and private enterprise data without expensive retraining.
- Grounded responses with direct source citations (substantially cuts hallucinations).
- Easy access control (filter retrieved chunks by user permissions).

• Drawbacks:

- "Garbage in, garbage out": Poor retrieval leads to incorrect or hallucinated answers.
- Adds latency (retrieval step + longer context processing).
- Requires additional infrastructure (embedding pipelines, vector databases, chunking strategies).

21. VECTOR DATABASE

- **Vector Database:** A specialized database optimized for storing, indexing, and querying high-dimensional vector embeddings with low latency (e.g., Pinecone, Milvus, Chroma, Qdrant).

• ★ Vector DB vs Traditional Database:

- *Traditional DB (SQL):* Searches on **exact matches** or scalar logic (`WHERE age > 25`).
- *Vector DB:* Searches on **mathematical semantic similarity** ("Find the 5 documents closest in meaning to this query").

• Similarity Measures:

- *Cosine Similarity:* Measures the angle between vectors (direction only, ignoring magnitude). Standard for NLP embeddings.
- *Dot Product:* Multiplies components; combines direction and magnitude.
- *Euclidean Distance (L_2):* Measures straight-line geometric distance.

- ★ **Approximate Nearest Neighbor (ANN):** Fast indexing algorithms (e.g., HNSW) that trade a tiny fraction of search accuracy for massive speedups across millions of vectors.

22. AI AGENTS

- **AI Agent:** An autonomous system that uses an LLM as its reasoning engine, combined with planning, memory, and external tools to execute multi-step tasks to accomplish a goal.

- **Core Loop:** $\text{Observe State} \rightarrow \text{Reason / Plan} \rightarrow \text{Act (Call Tool)} \rightarrow \text{Observe Result} \rightarrow \text{Repeat until Goal is met}$.
 - ★ **LLM vs AI Agent:**
 - *LLM:* A passive text-in, text-out predictor. Cannot take actions or interact with the external world.
 - *AI Agent:* An active system using an LLM to make decisions, execute Python code, query databases, call APIs, and self-correct.
 - ★ **Agent Challenges & Drawbacks:**
 - *Latency & Cost:* Requires multiple sequential LLM calls, increasing execution time and API cost.
 - *Error Cascading:* An error in step 1 propagates and derails subsequent steps.
 - *Security Risks:* Autonomous tool execution requires sandboxing to prevent destructive actions.
-

23. MODEL CONTEXT PROTOCOL (MCP)

- **Model Context Protocol (MCP):** An open standard for securely connecting AI applications and agents to external tools, local files, APIs, and databases.
 - **Why it matters:** Eliminates custom point-to-point glue code. MCP acts as the **"USB-C standard for AI"** — build an MCP server once for a tool, and any MCP-compliant AI client can seamlessly use it.
-

24. CONTEXT ENGINEERING

- **What it is:** The practice of curating, formatting, and dynamically managing the information fed into an LLM's context window.
 - **Key Components:**
 - *System Prompts:* Defining persistent persona, behavioral rules, and output formatting.
 - *Dynamic Retrieval (RAG):* Injecting only the most relevant document chunks into context.
 - *Conversation Management:* Sliding-window truncation or summarization of old chat turns to stay within token limits.
-

25. REASONING / MODERN AI CONCEPTS

- **Reasoning Models:** Models optimized (via RL and inference scaling) to generate extended internal chains of thought to explore multiple reasoning paths and self-correct before giving an answer (e.g., OpenAI o1/o3, DeepSeek-R1).
 - **Multimodal Models:** Models capable of processing and generating multiple data modalities simultaneously (Text, Images, Audio, Video).
 - **Small Language Models (SLMs):** Highly optimized compact models (1B to 8B parameters) designed to run efficiently on edge devices with minimal latency.
-

26. MODEL OPTIMIZATION

- **Knowledge Distillation:** Training a compact "Student" model to mimic the probability distributions and reasoning of a large "Teacher" model.

- **Quantization:** Lowering the numerical bit precision of model weights (e.g., 32-bit FP32 $\rightarrow$ 16-bit FP16 $\rightarrow$ 8-bit INT8 $\rightarrow$ 4-bit INT4), drastically reducing memory and compute requirements with minimal quality loss.
- **★ Distillation vs Quantization:**
 - *Distillation:* Changes the model architecture (creates a new smaller model).
 - *Quantization:* Keeps the exact same architecture and parameter count, but stores each parameter using fewer bits.

27. COMMON ML/DL TOOLS

Tool	Primary Purpose
NumPy	High-performance multi-dimensional array operations and scientific computing.
Pandas	Tabular data manipulation, cleaning, and DataFrame analysis.
Matplotlib & Seaborn	Data visualization, metric charting, and statistical plotting.
Scikit-learn	Classical ML algorithms, preprocessing pipelines, and evaluation metrics.
PyTorch	Leading open-source deep learning framework with dynamic computation graphs.
TensorFlow / Keras	Google's deep learning platform and high-level neural network API.
Hugging Face	Central hub for pretrained Transformer models, datasets, and tokenizers.
LangChain	Framework for developing LLM-powered applications (RAG, chains, agents).

28. THE COMPLETE ML PIPELINE

1. Problem Formulation

▼

2. Data Collection

▼

3. Exploratory Data Analysis

▼

4. Data Preprocessing

▼

5. Feature Engineering

▼

6. Data Splitting

▼

7. Baseline & Model Selection

▼

8. Training & Hyperparam Tuning

▼

9. Model Evaluation

▼

10. Deployment & Inference

→ Define objective, task type, and success metric

→ Gather raw data from databases, APIs, or files

→ Understand distributions, outliers, and correlations

→ Clean missing values, handle noise, encode categories

→ Construct new features; scale numerical values

→ Split into Train / Validation / Test sets (or K-Fold)

→ Establish simple baseline; train candidate models

→ Minimize loss via Gradient Descent; tune hyperparameters

→ Final evaluation on untouched Test set (F1, AUC, RMSE)

→ Deploy model as an API service for real-world predictions

[!TIP]

Top 10 Interview Distinctions Checklist:

1. **AI vs ML vs DL:** Scope hierarchy (AI = umbrella, ML = learns from data, DL = deep neural nets).
2. **Supervised vs Unsupervised vs RL:** Data signals (Labeled data vs Unlabeled patterns vs Reward signals).
3. **Feature vs Label:** Inputs (\$X\$) vs Target answer (\$y\$).
4. **Parameter vs Hyperparameter:** Learned by model (weights) vs Configured by engineer (learning rate).
5. **Loss vs Metric:** Internal differentiable optimization objective vs Human evaluation indicator.
6. **Precision vs Recall:** Cost of false alarms vs Cost of dangerous misses.
7. **Bagging vs Boosting:** Parallel variance reduction (Random Forest) vs Sequential bias reduction (XGBoost).
8. **CNN vs RNN vs Transformer:** Spatial grids vs Sequential loops vs Parallel attention.
9. **RAG vs Fine-Tuning:** Dynamic external knowledge retrieval vs Internal parameter updating.
10. **LLM vs AI Agent:** Passive text predictor vs Active autonomous goal-oriented loop.